

Practica: Patron Memento con TypeScript

En esta practica vamos a construir un editor simple que puede guardar estados, deshacer cambios y rehacerlos usando el patron Memento.

La idea central es separar tres responsabilidades: quien cambia el estado, quien guarda una copia y quien administra el historial.

Paso 1: definir el estado

Crea el tipo que representa la nota editable

```
type NoteState = {  
  title: string;  
  body: string;  
  tag: "idea" | "task" | "archive";  
};  
  
type NoteSnapshot = NoteState & {  
  id: number;  
  label: string;  
};
```

`NoteState` es el estado real. `NoteSnapshot` agrega datos para identificar cada guardado dentro del historial.

Paso 2: crear el Memento

El Memento conserva una copia del estado

```
class NoteMemento {  
  constructor(private readonly snapshot: NoteSnapshot) {}  
  
  getState(): NoteSnapshot {  
    return structuredClone(this.snapshot);  
  }  
}
```

La copia se devuelve con `structuredClone` para evitar que otra parte del programa modifique directamente el snapshot guardado.

Paso 3: crear el Originator

El Originator modifica, guarda y restaura su propio estado

```
class NoteOriginator {
  private state: NoteState;

  constructor(initialState: NoteState) {
    this.state = initialState;
  }

  update(nextState: NoteState) {
    this.state = nextState;
  }

  save(label: string): NoteMemento {
    return new NoteMemento({
      ...this.state,
      id: Date.now(),
      label,
    });
  }

  restore(memento: NoteMemento): NoteState {
    const { id, label, ...state } = memento.getState();
    this.state = state;
    return this.state;
  }
}
```

Paso 4: crear el Caretaker

El Caretaker administra las pilas de undo y redo

```
class HistoryCaretaker {
  private past: NoteMemento[] = [];
  private future: NoteMemento[] = [];

  push(memento: NoteMemento) {
    this.past.push(memento);
    this.future = [];
  }

  undo(current: NoteMemento): NoteMemento | null {
    const previous = this.past.pop();
    if (!previous) return null;

    this.future.unshift(current);
    return previous;
  }

  redo(): NoteMemento | null {
    const next = this.future.shift();
    if (!next) return null;

    this.past.push(next);
    return next;
  }
}
```

El Caretaker no cambia el contenido del Memento. Solo mueve snapshots entre la pila de deshacer y la pila de rehacer.

Paso 5: probar el flujo

Simula cambios desde consola

```
const originator = new NoteOriginator({
  title: "Nota inicial",
  body: "Primer contenido",
  tag: "idea",
});

const caretaker = new HistoryCaretaker();

caretaker.push(originator.save("Estado inicial"));

originator.update({
  title: "Nota editada",
```

```

    body: "Contenido modificado",
    tag: "task",
  });

  caretaker.push(originator.save("Estado editado"));

  const current = originator.save("Estado actual");
  const previous = caretaker.undo(current);

  if (previous) {
    const restored = originator.restore(previous);
    console.log("Restaurado:", restored);
  }

```

Paso 6: preguntas de cierre

- Que clase actua como Originator?
- Que clase actua como Memento?
- Por que el Caretaker no deberia modificar directamente el estado guardado?
- Que pasa con la pila de redo cuando se guarda un nuevo estado?

Reto

Restaurar un punto especifico del historial

Agrega un metodo `restoreAt(index)` dentro de `HistoryCaretaker`. Este metodo debe recibir la posicion de un snapshot guardado, devolver ese Memento y mover todos los estados que estaban despues de esa posicion a la pila de redo.

Reglas:

- Si el indice no existe, debe devolver `null`.
- El estado seleccionado debe quedar como el ultimo elemento de `past`.
- Los estados posteriores deben pasar a `future`.

Solucion del reto

Metodo `restoreAt(index)`

```

class HistoryCaretaker {
  private past: NoteMemento[] = [];
  private future: NoteMemento[] = [];

  push(memento: NoteMemento) {
    this.past.push(memento);
    this.future = [];
  }

```

```
}

undo(current: NoteMemento): NoteMemento | null {
  const previous = this.past.pop();
  if (!previous) return null;

  this.future.unshift(current);
  return previous;
}

redo(): NoteMemento | null {
  const next = this.future.shift();
  if (!next) return null;

  this.past.push(next);
  return next;
}

restoreAt(index: number): NoteMemento | null {
  const selected = this.past[index];
  if (!selected) return null;

  const removed = this.past.splice(index + 1);
  this.future = [...removed, ...this.future];

  return selected;
}
}
```

Con esta solución, el Caretaker sigue sin modificar el contenido de los Mementos. Solo reorganiza las pilas para que el historial pueda volver a un punto específico.